

Socket Programming

Socket Programming in Python  (<https://realpython.com/python-sockets/>)

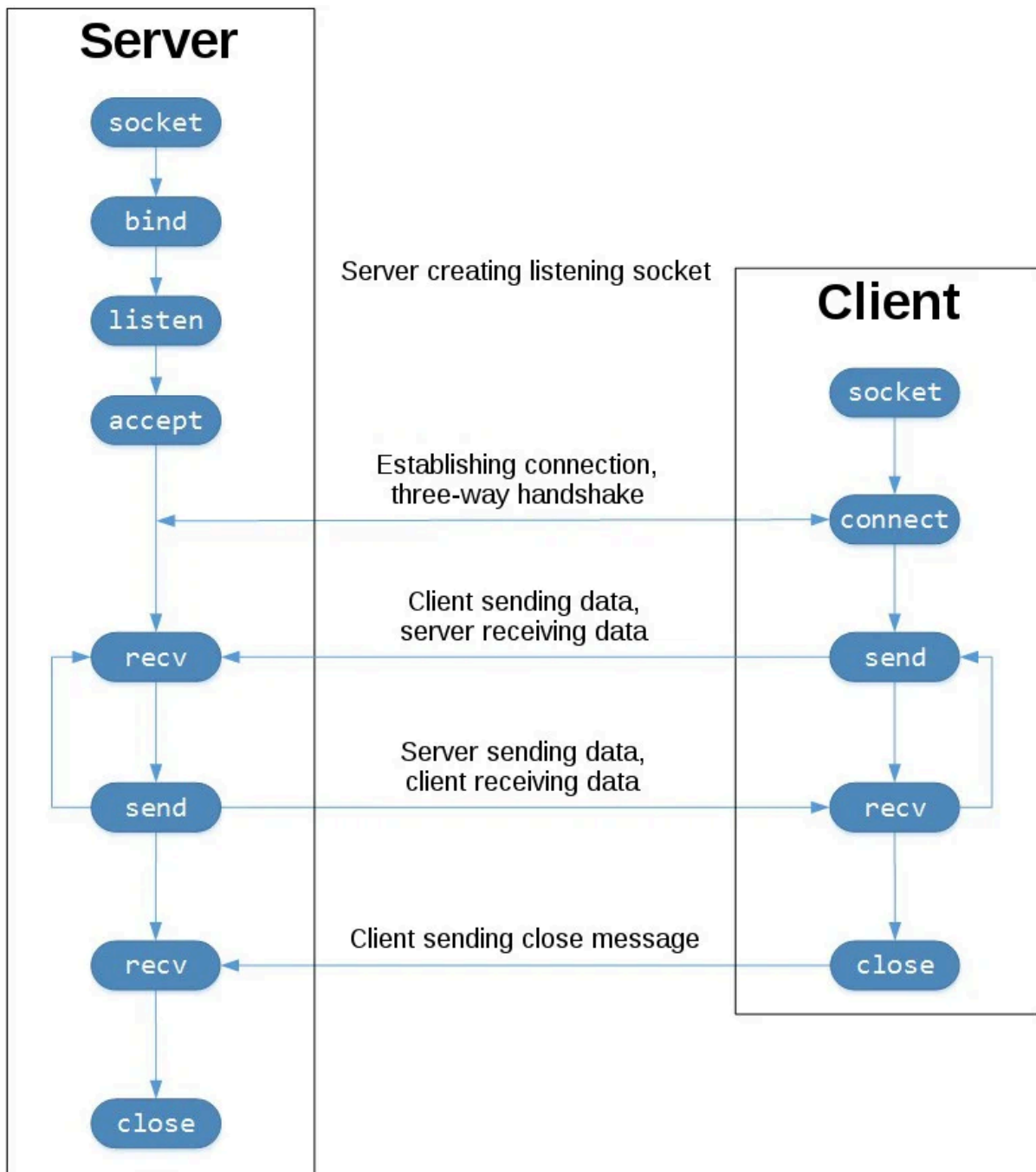
A **socket** is a communication endpoint in a computer network that allows two computers to communicate with each other. It is a software construct that allows a computer to send and receive data over the network.

For example, consider a client computer that wants to send a request to a server to access a webpage. The client creates a socket and sends the request to the server through the socket. The server receives the request and sends a response back to the client through the same socket. This allows the two computers to communicate with each other and exchange data.

Sockets and the socket API are used to send messages across a network. They provide a form of inter-process communication (IPC).

Inter-process communication (IPC), are the mechanisms provided by an operating system for processes to manage shared data.

You're going to create a socket object using `socket.socket()`, specifying the socket type as `socket.SOCK_STREAM`. When you do that, the default protocol that's used is the Transmission Control Protocol (TCP).



Server

`socket.socket()` creates a socket object that supports the [context manager type](https://docs.python.org/3/reference/datamodel.html#context-managers) [↗](https://docs.python.org/3/reference/datamodel.html#context-managers) (<https://docs.python.org/3/reference/datamodel.html#context-managers>), so you can use it in a `with` statement [↗](https://docs.python.org/3/reference/compound_stmts.html#with) (https://docs.python.org/3/reference/compound_stmts.html#with). There's no need to call `s.close()`:

The arguments passed to `socket()` [↗](https://docs.python.org/3/library/socket.html#socket.socket) (<https://docs.python.org/3/library/socket.html#socket.socket>) are [constants](https://docs.python.org/3/library/socket.html#constants) [↗](https://docs.python.org/3/library/socket.html#constants) (<https://docs.python.org/3/library/socket.html#constants>) used to specify the [address family](https://realpython.com/python-sockets/#socket-address-families) [↗](https://realpython.com/python-sockets/#socket-address-families) (<https://realpython.com/python-sockets/#socket-address-families>) and socket

type. `AF_INET` is the Internet address family for **IPv4** [↗\(https://en.wikipedia.org/wiki/IPv4\)](https://en.wikipedia.org/wiki/IPv4). `SOCK_STREAM` is the socket type for **TCP** [↗\(https://realpython.com/python-sockets/#tcp-sockets\)](https://realpython.com/python-sockets/#tcp-sockets), the protocol that will be used to transport messages in the network.

The `.bind()` method is used to associate the socket with a specific network interface and port number.

`.listen()` enables a server to accept connections. It makes the server a “listening” socket.

The `.accept()` [↗\(https://docs.python.org/3/library/socket.html#socket.socket.accept\)](https://docs.python.org/3/library/socket.html#socket.socket.accept) method **blocks** [↗\(https://realpython.com/python-sockets/#blocking-calls\)](https://realpython.com/python-sockets/#blocking-calls) execution and waits for an incoming connection. When a client connects, it returns a new socket object representing the connection and a tuple holding the address of the client. The tuple will contain `(host, port)` for IPv4 connections.

One thing that’s imperative to understand is that you now have a new socket object from `.accept()`. This is important because it’s the socket that you’ll use to communicate with the client. It’s distinct from the listening socket that the server is using to accept new connections.

After `.accept()` provides the client socket object `conn`, an infinite `while` **loop** [↗\(https://realpython.com/python-while-loop/\)](https://realpython.com/python-while-loop/) is used to loop over **blocking calls** [↗\(https://realpython.com/python-sockets/#blocking-calls\)](https://realpython.com/python-sockets/#blocking-calls) to `conn.recv()`. This reads whatever data the client sends and echoes it back using `conn.sendall()`.

If `conn.recv()` returns an empty `bytes` [↗\(https://docs.python.org/3/library/stdtypes.html#bytes-objects\)](https://docs.python.org/3/library/stdtypes.html#bytes-objects) object, `b''`, that signals that the client closed the connection and the loop is terminated.

The `with` **statement** [↗\(https://realpython.com/python-with-statement/\)](https://realpython.com/python-with-statement/) is used with `conn` to automatically close the socket at the end of the block.

Client

In comparison to the server, the client is pretty simple. It creates a socket object, uses `.connect()` [↗\(https://docs.python.org/3/library/socket.html#socket.socket.connect\)](https://docs.python.org/3/library/socket.html#socket.socket.connect) to connect to the server and calls `s.sendall()` to send its message. Lastly, it calls `s.recv()` to read the server’s reply and then **prints it** [↗\(https://realpython.com/python-print/\)](https://realpython.com/python-print/).